
Lean-ing into polynomials:

A Computer's Guide to Mathematical Proofs

Math Directed Reading Project

Toyosi Abu

Mentor: Jin Wei

Table of contents

01

Systems of
Polynomial Equations

02

Lean 101

03

Implementation

04

Future Work

01

Solving systems of Polynomials

Univariate vs Multivariate Polynomials

Univariate = Easy

$$x - y + z = 2$$

$$2x - y - z = -6$$

$$2x + y + 2z = -3$$

$(-2, -1, 3)$

Multivariate = Hard (on paper)

$$3x^4y^2 - 7xy^4z = -80$$

$$12z - 39xz^2 = 10$$

$$41y^3z^4 + 2xz = 15$$

$(?, ?, ?)$

The Elimination Polynomial Theorem

System of Polynomials

$$P_1(x_1, \dots, x_d) = 0$$

$$\vdots$$
$$\vdots$$

$$P_n(x_1, \dots, x_d) = 0$$



Elimination Polynomial f

$\exists f \in \mathbb{Q}[x_d]$ with roots $b_1, \dots, b_m$
s.t some power of $f \in \langle P_1, \dots, P_n \rangle$

All Values of x_d in our Solution Set

$$b_1, \dots, b_m$$

The Elimination Polynomial In Action

System of Polynomials

$$P_1(x_1, \dots, x_d) = 0$$

$$P_2(x_1, \dots, x_d) = 0$$

$$P_3(x_1, \dots, x_d) = 0$$

$$P_4(x_1, \dots, x_d) = 0$$

$$x_d = \text{some } b_i \text{ (say } b_3)$$

System of Polynomials'

$$P_1(x_1, \dots, b_3) = 0$$

$$P_2(x_1, \dots, b_3) = 0$$

$$P_3(x_1, \dots, b_3) = 0$$

$$P_4(x_1, \dots, b_3) = 0$$

Proof

Recall roots of $f(x_d) = \{b_{1..m}\}$

$$f^k \in \langle P_1, \dots, P_n \rangle = \sum_{i=1}^n f_i P_i(x^-)$$

$$x^- = \{x_1, \dots, x_d\}$$

$$\sum_{j=1}^m f_i P_i(x^- | b_j) = 0$$

The Elimination Polynomial In Action

Repeat the elimination step $d-1$ times

Now Univariate (only x_1 to solve)

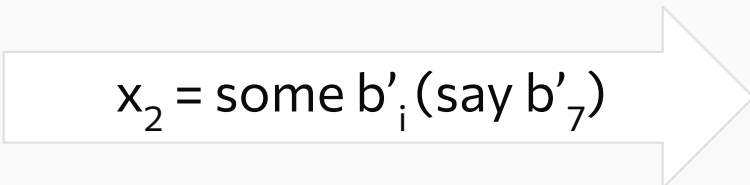
System of Polynomials'

$$P_1(x_1, \dots, b_3) = 0$$

$$P_2(x_1, \dots, b_3) = 0$$

$$P_3(x_1, \dots, b_3) = 0$$

$$P_4(x_1, \dots, b_3) = 0$$


$$x_2 = \text{some } b'_i \text{ (say } b'_7)$$

System of Polynomials''

$$P_1(x_1, b'_7, \dots, b_3) = 0$$

$$P_2(x_1, b'_7, \dots, b_3) = 0$$

$$P_3(x_1, b'_7, \dots, b_3) = 0$$

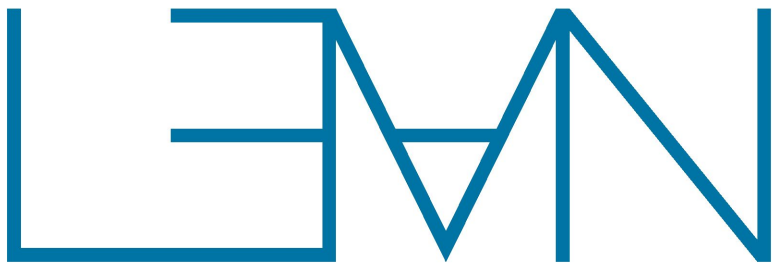
$$P_4(x_1, b'_7, \dots, b_3) = 0$$

02

Lean 101

And how would a computer do
this proof?

What is Lean?



An open-source functional programming language and interactive theorem prover.

- Functional Programming Language
Similar to OCaml, Haskell
- Proofs as programs
Curry-Howard Isomorphism
- Verification through compilation

How do we do proofs? With Mathlib!

Mathlib.Analysis.Calculus.LocalExtr.Rolle



theorem exists_deriv_eq_zero

[source](#)

```
{f : ℝ → ℝ} {a b : ℝ} (hab : a < b)
(hfc : ContinuousOn f (Set.Icc a b)) (hfI : f a = f b) :
  ∃ c ∈ Set.Ioo a b, deriv f c = 0
```

Rolle's Theorem `deriv` version

theorem exists_hasDerivAt_eq_zero'

[source](#)

```
{f f' : ℝ → ℝ} {a b l : ℝ} (hab : a < b)
(hfa : Filter.Tendsto f (nhdsWithin a (Set.Ioi a)) (nhds l))
(hfb : Filter.Tendsto f (nhdsWithin b (Set.Iio b)) (nhds l))
(hff' : ∀ x ∈ Set.Ioo a b, HasDerivAt f (f' x) x) :
  ∃ c ∈ Set.Ioo a b, f' c = 0
```

Rolle's Theorem, a version for a function on an open interval: if f has derivative f' on (a, b) and has the same limit l at $\mathcal{N}[>] a$ and $\mathcal{N}[<] b$, then $f' c = 0$ for some $c \in (a, b)$.

03

Implementation of the Elimination Polynomial

Challenges in Defining Roots

Very Limited Support for Computational Polynomials

```
example (i :  $\sigma$ ) : MvPolynomial  $\sigma$  R := X i

-- A few definitions for the roots of a given MvPolynomial. TEMP : Ignoring Multiplicity for now, alternative definition for rootSet

-- def rootMultiplicity (a :  $\sigma \rightarrow R$ ) (p : MvPolynomial  $\sigma$  R) :  $\mathbb{N}$  :=
--   if  $\neg(\text{MvPolynomial.eval } a \text{ } p) = (0 : R)$  then
--     0
--   else 1 + rootMultiplicity a (p | a)

-- theorem exists_multiset_roots {p : MvPolynomial  $\sigma$  R} ( $\_ : p \neq 0$ ) :  $\exists s : \text{Multiset } (\sigma \rightarrow R)$ ,
--   ( $\text{Multiset.card } s : \text{WithBot } \mathbb{N}$ )  $\leq$  totalDegree p  $\wedge \forall a, s.\text{count } a = \text{rootMultiplicity } a \text{ } p :=$ 
--   sorry

-- def roots (p : MvPolynomial  $\sigma$  R) : Multiset ( $\sigma \rightarrow R$ ) :=
--   haveI := Classical.decEq R
--   haveI := Classical.dec (p = 0)
--   if h : p = 0 then  $\emptyset$  else Classical.choose (exists_multiset_roots h)

-- def rootSet (p : MvPolynomial  $\sigma$  R) : Set ( $\sigma \rightarrow R$ ) := {x | x  $\in$  roots p}

-- Note: Multiplicity might be really hard to implement: https://www.researchgate.net/publication/382970870\_First\_steps\_towards\_Computational\_Polynomial\_Roots

def IsRoot (a :  $\sigma \rightarrow R$ ) (p : MvPolynomial  $\sigma$  R) : Prop := (MvPolynomial.eval a p) = (0 : R)

def rootSet (p : MvPolynomial  $\sigma$  R) : Set ( $\sigma \rightarrow R$ ) := {x | IsRoot x p}
```

Univariates as Sub Semiring

```
def univariate_polynomial (R : Type u) [CommRing R] (d :  $\sigma$ ) : Subsemiring (MvPolynomial  $\sigma$  R)
  where
  carrier := {p : MvPolynomial  $\sigma$  R |  $\forall$  (v1 v2 :  $\sigma \rightarrow R$ ), (v1 d = v2 d)  $\rightarrow$  (MvPolynomial.eval v1 p = MvPolynomial.eval v2 p)}
  zero_mem' := by
    intros v1 v2
    simp [MvPolynomial.eval_zero]

  one_mem' := by
    intros v1 v2
    simp [MvPolynomial.eval_C]

  add_mem' := by
    intros p q hp hq v1 v2 h
    simp [MvPolynomial.eval_add]
    rw [hp v1 v2 h, hq v1 v2 h]

  mul_mem' := by
    intros p q hp hq v1 v2 h
    simp [MvPolynomial.eval_mul]
    rw [hp v1 v2 h, hq v1 v2 h]
```

▼ Tactic state

1 goal

```
R : Type u
inst1 : CommRing R
inst2 : IsDomain R
inst3 : DecidableEq R
 $\sigma$  : Type v
inst4 : DecidableEq ( $\sigma \rightarrow R$ )
R : Type u
inst5 : CommRing R
d :  $\sigma$ 
p q : MvPolynomial  $\sigma$  R
hp : p ∈
  { carrier := {p |  $\forall$  (v1 v2 :  $\sigma \rightarrow R$ ), v1 d = v2 d  $\rightarrow$  (eval v1) p = (eval v2) p}, mul_mem' := ..., one_mem' := ... }.carrier
hq : q ∈
  { carrier := {p |  $\forall$  (v1 v2 :  $\sigma \rightarrow R$ ), v1 d = v2 d  $\rightarrow$  (eval v1) p = (eval v2) p}, mul_mem' := ..., one_mem' := ... }.carrier
v1 v2 :  $\sigma \rightarrow R$ 
h : v1 d = v2 d
⊢ (eval v1) p + (eval v1) q = (eval v2) p + (eval v2) q
```

► All Messages (0)

11

Future Work

Full Elim Theory

Expanding on current work to finish off the elimination polynomial theorem

Improve Mathlib

Share implemental changes to the Lean/Mathlib to make updates



Gröbner Bases

Creating definition to complete the system of polynomials

Thank You!

Professor Pemantle and Julian - Chapter 3 (Systems of Polynomial Equations) of DRP Book

Léo and Maxine - Running the DRP Program

Jin - Fantastic Mentor!!!!
